ChatGPT

Executive Summary

We have identified a **shifted patent landscape** emerging from the repository: the key inventions center on *state admission* and *consequence gating*, not just provenance. In particular, the critical new patent nuclei are concrete computational engines inserted between state reconstruction and execution:

- **Typed State Admission Engine (Tier 1):** A process that takes a reconstructed candidate state, evaluates a suite of typed predicates (authority, temporal, tracking, etc.), and produces an *authoritative runtime state* only if all predicates pass (fail-closed otherwise).
- **Replay-Time Re-Admission Engine (Tier 1):** On each replay (or recovery), the system re-applies the admission engine to verify the state under current context (e.g. changed authority or time), rather than blindly trusting the stored snapshot.
- **Consequence-Indexed Identifiability Gate (Tier 1):** A mechanism that builds the **observational-equivalence class** of all worlds consistent with admitted observations, independently derives each world's consequences, and **blocks execution** of any consequence-affecting transition unless all worlds yield the same consequence. In other words, execution is *withheld* if the admitted observation class implies multiple lawful futures (STATE/C ⇒ fixed).
- **Consequence-Tag Selection Engine (Tier 1):** Annotates the admitted state with *consequence tags* and uses them to classify transitions (e.g. "semantic change", "discharge", etc.), driving execution choice.
- **Typed Failure Receipt System (Tier 1):** Every admission or transition failure produces a structured receipt (not just a log entry) containing the predicate type, evidence, rule, and required reopening, in a form that participates in execution logic (not just logging).

Secondary inventions (Tier 2) include: a **Consequence-Preserving Transition Validator** (ensuring each admitted distinction is assigned exactly one disposition), **Disposition-Closure Computation**, **Reopening-Propagation Engine**, and **Authority Graph Engine**. Broad "constitutional" concepts (runtime constitution, etc.) are best left as research topics.

These inventions transform the pipeline into:

```
flowchart LR
    A[Observed Repository State] --> B[Reconstructed Candidate State]
    B --> C[Replay-Time Re-Admission]
    C --> D[Typed Admission Engine]
    D --> E[Authoritative Runtime State]
    E --> F[Consequence Derivation (Independent Futures)]
    F --> G[Identifiability Gate]
    G --> H[Selected Consequence]
    H --> I[Transition Execution]
```

Each nucleus can be claimed as a **specific technical improvement** to computer state and execution logic (per USPTO guidance【20†L1-L4】【18†L23-L27】), using well-defined inputs, data flows, and concrete algorithms. The report below prioritizes these inventions (Tier 1 highest) and details for each: claim sketches, inputs/outputs, core steps, data structures, semantics, pseudocode outline, complexity, prior-art to review, and drafting strategies. We also recommend which families to include in a provisional filing, map them to repository gaps, assess patentability risks, and propose engineering tasks to build enabling embodiments. All claims should emphasize explicit predicates, receipts, and algorithms, following USPTO guidance to recite improvements to "the underlying computing device" (e.g. efficiency, security)【20†L1-L4】【23†L64-L67】.

1. Tier 1 Nuclei (High Priority)

1.1. Typed State Admission Engine

- **Claim Frame:** *"A computer-implemented method (or system) for authoritative state admission, comprising: receiving a reconstructed candidate state; evaluating a set of typed admission predicates on the candidate state; and, only if all predicates pass, transforming the candidate state into an authoritative runtime state; wherein each predicate is of a specific type (e.g. byte-identity, source-existence, tracked-state, authority, temporal, non-interference, canon-eligibility) and produces a receipt."*

- **Inputs:** Reconstructed state (bytes, object graph, metadata), predicate definitions (e.g. public keys, manifest of tracked objects, time window).

- **Outputs:** Either an *AuthoritativeState* object (when admitted) or a failure signal with receipts; each predicate's outcome and evidence is recorded.
- **Core Steps:**
- **Partition** the candidate state into tracked vs untracked elements (per system's schema).
- For each **typed predicate** (authority key match, timestamp check, source existence, etc.):
    - Independently evaluate on the candidate state.
    - Produce a typed *predicate receipt* containing status, evidence, failure mode, and rules for reopening/replay.
- **Combine** predicate results: if any fail (or evidence missing), terminate and reject the state (fail-closed).

- If all pass, emit an **Admission Receipt** and construct the *Authoritative Runtime State* from the candidate.

- **Data Structures:**

- *AdmissionReceipt* (recording each predicate, evidence, verdict).
- *TypedPredicate* definitions (enums or classes for BYTE_IDENTITY, etc.).

- CandidateState and AuthoritativeState objects (could be versions of the same class, with flags).

- **Execution Semantics:** Fail-closed (no assumption of truth without predicates). This is a **technical barrier** protecting the runtime from unverified state. Admission is stronger than mere hash-check: it checks authoritative provenance and policy compliance.

- **Minimal Embodiment (Pseudocode):**

```
class AdmissionEngine:
    def admit(candidateState):
        receipts=[]
        for pred in [BYTE_ID, SOURCE_EXISTS, TRACKED, ...]:
            result, evidence = pred.evaluate(candidateState)
            receipts.append(PredicateReceipt(pred.name, result, evidence))
            if result == FAIL:
                return AdmissionFailure(receipts)
        return AdmissionSuccess(candidateState, receipts)
```

Complexity: O(p × verification) where p = number of predicates, plus state partitioning. In practice, predicates often index into logs or use digests (e.g. O(n) byte scan for integrity).

- **Prior Art & Keywords:**
- **W3C PROV**: *Provenance* frameworks (PROV-DM【23†L64-L67】) record data lineage but do not enforce admission or predicate gating. PROV is *domain-agnostic*, no built-in "admission" concept.
- **In-toto**: supply-chain integrity tool (ensures steps are carried out in order【24†L8-L11】) – analog: each state admission step is like a supply-chain check.
- **ZCAP (Capability-based Auth)**: authorization specs (e.g. W3C ZCAP-LD) cover rights but not operational admission algorithms.
- **State checkpointing** literature (look up "restart veto predicate", "process checkpoint recovery"); (historic OS / transaction recovery do checks but not typed, receipts).

- **Queries:** "state admission predicates", "secure state restoration", "trusted snapshot verification".

- **Claim Drafting Tips:** Emphasize *computer-specific operations* (hashing bytes, certificate checks, executing predicates). Avoid abstract terms like "compliance"; instead say "evaluating digital signatures and tracking metadata." Cite improvement in computer **security and reliability**, per USPTO: e.g. how admission reduces CPU cycles wasted on invalid states or stops unauthorized code execution.

1.2. Replay-Time Re-Admission

- **Claim Frame:** *"A method of replaying a recorded system state, comprising: reconstructing a candidate state from stored data; re-evaluating admission predicates on the reconstructed state under current context to determine a new admission result; and proceeding with execution only if re-admitted."*

- **Inputs:** Stored snapshot (from disk/ledger), current environment (key-set, time, configuration).

- **Outputs:** Same as above: either confirmed authoritative state (and possibly updated receipts) or rejection.
- **Core Idea:** *Every* replay (or recovery from persistent state) invokes the admission engine again. The system *does not* assume that a past-verified snapshot is still valid. This can catch situations like changed revocations, time expirations, or new trust policies.
- **Steps:**
- Load snapshot to *CandidateState*.
- Invoke **Typed State Admission Engine** (as above) on candidate.
- If admission fails, abort replay or require additional evidence (perhaps rollback or request missing keys).
- **Data Structures:** Same as Admission Engine; plus store *AdmissionReceipts* from initial load for comparison.
- **Semantics:** Ensures *consistency* over time. Conceptually, **Verified Snapshot ≠ Authoritative State** without re-admission.
- **Minimal Embodiment:**

```
def replay(snapshotFile, currentTime):
    candidate = load(snapshotFile)
    admission = AdmissionEngine.admit(candidate)
    if admission.failed:
        raise ReplayError("State admission failed", admission.receipts)
    return candidate   # now authoritative
```

Complexity: Extra cost = running admission on each replay (cost same as admission engine, typically linear in snapshot size).

- **Prior Art & Keywords:**
- **Database Recovery**: ARIES protocol, “redo/undo” may recompute state; not typically do full semantic checks.
- **Persistent State Machine**: checkpoint validation, PBFT replay logic (some systems *do* verify signatures on messages in log but may assume state reconstruction is valid).
- **Active Re-verification**: search terms “runtime re-verification of snapshot”, “replay integrity check”, “bootstrap state authorization”.
- **Claim Tips:** Stress that re-admission is a **technical process** of recomputing digest/predicate status, not an abstract policy. Emphasize solving problems like stale keys or untracked data sneaking in. Possibly tie to metrics: e.g. prevents a vulnerability exploited by resuming an old state (improvement in reliability).

1.3. Consequence-Indexed Identifiability Gate

- **Claim Frame:** *"A method of gating execution of a transition based on partial observations, comprising: computing the observational-equivalence class of all admitted possible worlds given the current observations; independently deriving a proposed consequence (future state or effect) for each world; and permitting the transition to execute only if all derived consequences are identical; otherwise, withholding execution until a refinement observation is acquired."*

- **Inputs:** Admitted state and planned transition. The state includes an **Observation** vector (values of variables or receipts from the admission stage). The transition has an associated **Consequence function** C (maps worlds to next-state effect).

- **Outputs:** Either a single admitted *consequence* (if IDENTIFIABLE) or a decision to withhold, plus a recommendation of next observation.
- **Core Steps:**
- **Build observational equivalence:** Let O be the projection of admitted state on observations. Compute $E = \{R : O(R) = O\}$, the set of possible worlds consistent with the observed state.
- **Consequence derivation:** For each world $R \in E$, compute $C(R)$ independently **using only the admitted basis** B_R. That is, derive consequences without assuming the truth of the outcome.
- **Anti-circularity check:** Ensure derivations do not causally depend on the outcome itself (counterfactual test). E.g. swap labels or intervene and verify $B_R \not\rightsquigarrow O$.
- **Identifiability decision:** If $\{C(R) : R \in E\}$ is a singleton set $\{c\}$, then consequence is *IDENTIFIABLE* = c. Otherwise, mark it *NOT_IDENTIFIABLE*.

- If *NOT_IDENTIFIABLE*, **inhibit** the transition and optionally select the next observation to reduce uncertainty (see next engine).

- **Data Structures:**

- *ObservationClass* (set or representation of equivalent worlds, typically implicit).
- *ConsequenceBasis* with dependency DAG.
- *IdentifiabilityReceipt*: records observation key, images of C, status (IDENTIFIABLE, NOT_ID, INDEP_UNRESOLVED).

- **Execution Semantics:** This gate intervenes *at execution time* of a transition, so it is critical to system correctness. Execution proceeds (with consequence c) only if all admitted worlds yield c; otherwise the system must *wait* or ask for more information.

- **Minimal Embodiment (Pseudocode):**

```
def check_consequence_identifiability(admittedState, transition):
    E = admittedState.observational_equiv_class()
    consequences = set()
    for R in E:
        C_R = derive_consequence(transition, R)  # use basis B_R
        consequences.add(C_R)
```

```
    if len(consequences)==1:
        return IdentifiableResult(consequences.pop())
    else:
        return NotIdentifiableResult(consequences)
```

Complexity: Potentially exponential in the number of observations! In practice, we avoid enumerating E explicitly by symbolic reasoning or by selecting distinguishing queries. The claim need not require brute force; it's enough to describe the test and gating.

- **Prior Art & Keywords:**
- **Runtime Verification (monitoring):** e.g. three-valued semantics【2†L15-L18】 already allow inconclusive verdicts when trace is partial. Our "NOT_IDENTIFIABLE" extends this: it is not "unknown truth" but "multiple legal futures exist".
- **Causal Identifiability:** Work by Pearl et al. on which counterfactuals (consequences) can be determined from observations【9†L43-L49】. ("LATE is not identifiable from data" illustrates that without more info, effect is ambiguous).
- **Active Learning:** Selecting queries to split hypothesis (here: "which next observation reduces consequence entropy"). Algorithms like version-space halving【10†L172-L180】 underlie our query strategy.

- **Queries:** "observational equivalence class execution", "counterfactual gating runtime", "identifiable consequence partial info".

- **Claim Tips:** Emphasize the concrete loop of deriving *each world's* consequence with its *own basis* (dependency analysis). Avoid abstract wording like "sound classification"; instead call it "deriving transitions under each compatible hypothesis" or "performing multiple speculative derivations". Highlight that consequences are invariant or not, rather than just saying "uncertain". If possible, tie to better CPU usage (not guessing wrong) or security (no unauthorized state splintering).

1.4. Consequence-Tag Selection Engine

- **Claim Frame:** *"A system for selecting execution actions using consequence tags, comprising: obtaining an admitted state annotated with one or more consequence tags (labels that categorize the semantic effect of transitions); grouping candidate transitions by their tags; and selecting transitions for execution based on the tags (e.g. priority, policy, or compatibility)."*

- **Inputs:** Authoritative state, each transition labeled with tags (e.g. "electronic equivalence", "discharge", "semantic override"). A tag vocabulary is defined by the system.

- **Outputs:** A chosen transition or execution path. Possibly an "execution plan" that preserves certain properties.
- **Core Steps:**
- After state admission, *assign tags* to distinctions in state or to scheduled transitions.
- Partition transitions into classes by tag (e.g. reversible vs. irreversible, structural vs semantic).
- Apply a **selection algorithm** (could be first-come, priority weights, or integrity check) to pick one representative from each class, or to resolve conflicts.

- Mark the selected transitions as approved for execution; others may be postponed or cancelled.

- **Data Structures:**

- *ConsequenceTagSet*: defines tags per transition.
- *Tag Index*: mapping state objects or transitions to tags (could be a multimap).

- *Policy Table*: rules for handling each tag.

- **Execution Semantics:** The engine **actually influences control flow** based on semantic tags. For example, it may ensure that only one of several equivalent "superseded" states is applied.

- **Minimal Embodiment:**

```
def select_by_tag(authoritativeState, transitions):
    by_tag = defaultdict(list)
    for t in transitions:
        tag = t.get_tag(authoritativeState)
        by_tag[tag].append(t)
    # Example policy: pick one from each tag class
    selected = []
    for tag, ts in by_tag.items():
        selected.append(policy_pick(tag, ts))
    return selected
```

Complexity: linear in number of transitions; tag lookup O(1).

- **Prior Art & Keywords:**
- **Database Query Optimization:** "query tags" and "operator selection" conceptually similar, but not directly relevant.
- **Event Routing Systems:** Tagging events for handlers (similar idea).

- **Literature:** Search "semantic scheduling tagged transitions", "policy-based action selection". (This seems novel in this context.)

- **Claim Tips:** Describe tags concretely (e.g. "financial settlement", "physical item transfer", "witness substitution"). Tie selection to computer state: e.g. "compute a transition priority table keyed by tag". Emphasize the system improves throughput or correctness by avoiding redundant or conflicting actions.

1.5. Typed Failure Receipt System

- **Claim Frame:** *"A method for generating structured receipts for execution failures, comprising: for each failed predicate or transition, creating a typed receipt object that includes the predicate type, evidence*

summary, failure classification, and reopening/replay rules; storing the receipt; and using the receipt contents to guide subsequent runtime behavior."

- **Inputs:** Failure events from admission or transition validation (including which predicate failed, error info).

- **Outputs:** A *FailureReceipt* object (possibly in JSON) that records details, and then triggers corrective actions.
- **Core Steps:**
- Upon any predicate or identifiability check failure, create a receipt with fields: (PredicateName, Evidence, FailureType, ReopenRule, ReplayRule, Timestamp).
- Store the receipt in a log or attach to state.

- The runtime decision logic reads receipts to decide: if "ReopenRule = true", then schedule a reopen of upstream state; if "ReplayRule=prompt", then abort and request new info.

- **Data Structures:**

- *FailureReceipt* schema (e.g. JSON with typed fields).

- Receipt log or attach to transaction context.

- **Execution Semantics:** Unlike simple logging, each receipt is **typed data** that participates in execution (for example, a failure might itself be admitted or replayed later).

- **Minimal Embodiment (Pseudocode):**

```
class PredicateReceipt:
    def __init__(self, name, status, evidence, reopen, replay):
        self.name=name; self.status=status; self.evidence=evidence
        self.reopenRule=reopen; self.replayRule=replay

def on_predicate_fail(pred, state):
    rec = PredicateReceipt(pred.name, 'FAIL', pred.evidence,
                           pred.reopen_rule, pred.replay_rule)
    log.append(rec)
    take_action(rec.reopenRule, rec.replayRule)
```

Complexity: Constant per failure.

- **Prior Art & Keywords:**
- **Runtime Logging**: traditional logs record errors, but not with structured "receipt" semantics.
- **Verifiable Logs**: schemes like Certificate Transparency provide logs, but not enforcement.

- **Formal Methods:** error witnesses or recovery certificates exist academically, but our receipts are execution objects.

- **Claim Tips:** Emphasize receipts as part of the **computer's decision logic** (e.g. "the receipt is parsed by the admission engine"), not just audit. Phrases like "typed evidence object" can help. Show how receipts improve recovery (an improvement in system resiliency).

2. Tier 2 Nuclei (Supporting/In-Progress)

These are promising but need prototype or more development. They can be provisional claims or described in spec:

- **Consequence-Preserving Transition Validator:** For each admitted distinction, ensure exactly one disposition. (E.g. "each address is either transformed or discharged, but not both.") This enforces a *closure invariant* on system actions. Requires tracking each object's fate.
- **Disposition Closure Computation:** The algorithm/invariant that the sum of dispositions equals number of distinctions (all admitted differences accounted for).
- **Reopening-Propagation Engine:** Given one predicate changes (e.g. authority revoked), compute which downstream consequences/states must be reopened. (Effectively, maintain a dependency graph and traverse it to identify affected tasks.)
- **Authority Graph Engine:** Build and maintain a graph of who-granted-what authorizations when; queries determine if current actions are allowed.
- **Evidence Surface Separation Engine:** Maintain separate "surfaces" of evidence (byte integrity, authorship, semantic truth, runtime validity). Prevent one surface's proof from implying another (e.g. valid hash does not imply valid authority).

These all deserve patent consideration but likely as dependent or narrower claims. They fit within the broader architecture and build on Tier 1.

3. Tier 3 Research Umbrellas (Do Not Patent at Present)

Concepts like "constitutional runtime," "meaning conservation," or the 8-way theory are too broad and not ready for claim scope. They should be pursued as research frameworks, not lead claims.

4. Prior Art References and Search Directions

For each nucleus, key prior-art domains and example queries:

- **Typed State Admission Engine:**
- *Sources:* W3C PROV (prov-dm【23†L64-L67】, prov-primer) – note PROV is domain-agnostic and has no admission logic.
- *Keywords:* "state restoration predicate validation", "trusted snapshot admission", "secure state reconstruction", "state checkpoint verification".
- *Others:* In-toto (supply-chain integrity【24†L8-L11】) – analogous but does not cover runtime re-admission. NIST Zero Trust architecture (ZCAP, Abac).

- *USPTO:* MPEP 2106.04(d) suggests specifying "technical improvement of underlying computer"【20†L7-L10】.

- **Replay-Time Re-Admission:**

- *Sources:* Database/transaction recovery (ARIES, but no semantic check); software fault tolerance (Belady's algorithm no).

- *Keywords:* "replay verification dynamic", "persisted state revalidation", "snapshot trust recalculation".

- **Consequence-Indexed Identifiability Gate:**

- *Sources:* Runtime verification (three-valued LTL【2†L15-L18】, monitoring incomplete traces). Causal inference literature – Pearl's do-calculus (identifiability)【9†L43-L49】; UAI papers on partial counterfactuals.

- *Keywords:* "partial observation execution gating", "compatibility class consequence determination", "counterfactual independence runtime".

- **Consequence-Tag Selection Engine:**

- *Sources:* Possibly none directly. Analogies in workflow management (task labels) or database triggers.

- *Keywords:* "policy-based action tagging", "semantic transition label", "execution scheduling with tags".

- **Typed Failure Receipts:**

- *Sources:* Logging and recovery mechanisms; technical specs like FIDO attestation, L2TP error codes. Not much formal.

- *Keywords:* "structured error receipts", "typed exception logs security", "formal recovery certificate".

- **Active Learning / Query Selection:**

- *Sources:* Active learning (pool-based, version space sampling【10†L172-L180】); experimental design (value of information).

- *Keywords:* "next best query active learning", "entropy-based query selection planning".

- **Causal Identifiability:**

- *Sources:* AAAI/NeurIPS on causal identification (see Maiti et al. 2025【9†L43-L49】, Shpitser & Pearl 2007).

- *Keywords:* "counterfactual identifiability algorithm", "what-if query identification".

- **USPTO Guidance:**

- *Sources:* MPEP §§2106–2106.05, 2106.04(d)(1) (improvements)【20†L1-L4】【20†L7-L10】.
- *Keywords:* “technical improvement patent eligibility”, “software patent claim improvement”.

5. Provisional Filing Strategy

Families to include:

- *Engine families:* One patent on **State Admission and Predicate Engine** (covering items 1.1, 1.2, parts of 1.5).
- **Identifiability gating** patent (covering 1.3, possibly 1.4 with tags, 1.5 on receipts as support).
- **Transition/Disposition** patent (covering validator, receipts, closure).
- Possibly one for **Min-Missing/Min-Sufficient Observation** logic (active learning).

Dependent claims: Each main claim above can have dependents for specific predicate types, receipts, data structures (e.g. “predicate returns a *Receipt object*【20†L1-L4】”), “transition classifier as a DAG traversal”, or integration details (authority graph, tag categories).

Prototypes before filing: Ensure a working prototype of 1.1–1.3. Concrete code demonstrating typed predicates (with receipts) and the identifiability gate. We need evidence that these aren’t obvious combinations.

6. Risk Assessment and Drafting

Family	Enablement Risk	Obviousness Risk	Prior-Art Risk	Strategy
Typed State Admission	Medium	Low	Medium	Emphasize new admission steps beyond known verification. Focus on concrete data flows (certificate checks) and outcomes. 【20†L1-L4】
Replay-Time Re-Admission	Low	Low	Low	Show difference from standard recovery; highlight dynamic re-check. Frame as practical improvement in safety.
Identifiability Gate	High (complex)	Medium	Medium	Provide algorithms (observation partition, consequence derivation). Claim concrete comparisons of worlds, not abstract “uncertainty.” Use dependency checking novelty.
Tag Selection Engine	Medium	High	High	May need strong demos. Tie to policy execution, speed improvement. Possibly combine with identifiable gating to strengthen novelty.

Family	Enablement Risk	Obviousness Risk	Prior-Art Risk	Strategy
Failure Receipts System	Low	Low	Low	Clearly distinguish from plain logs. Emphasize receipts as inputs to system logic.
Validator / Closure	Medium	Medium	Low	Formalize “exactly one disposition” invariant. Emphasize algorithm enforcing it.
Other broad concepts	High	High	High	**Not in claims.** Keep these conceptual, maybe provisional for background only.

Draft claims with at least one embodiment step (e.g., computing hashes, updating receipts) so they recite more than abstract ideas. For example, instead of “evaluate predicate,” say “compute a cryptographic digest and compare to expected, check digital signature, etc.”

7. Repository Gaps vs. Inventions (Mapping Table)

Repository Artifact	Gap / Issue	Invention	Required Code Changes
`state_reconstitution.py` (`StateLedger.latest()`)	*Trusts last JSONL row without re-checking.* No admission step.	Typed State Admission Engine; Replay-Time Re-Admission.	After loading JSON, call AdmissionEngine.admit(). If fail, abort. Do not simply trust stored digest or proceed.
`capsule.verification.json` (e.g. Brudo_OpenAI_Project_Knowledge_Capsule_v4)	Only performs byte-level verification; explicitly leaves semantics and authority “NOT_ESTABLISHED”.	Evidence Surface Separation; Authority Predicate Engine.	Introduce separate checks for authority (e.g., check KMS, keys), canonical status. Record independent evidence surfaces in receipts.
`replay` test suite (21 tests)	Current tests check basic independence, but did not differentiate IDENTIFIABLE vs NOT.	Replay-Time Re-Admission; Identifiability Gate.	Add tests for switching authority/context and for consequence-blocking on non-identifiability.
Domain model (tracked vs untracked)	Untracked bytes were ignored in proofs, but no formal predicate.	Track-state Predicate in Admission.	Implement explicit partition of state into “tracked” fields and enforce missing-tracked-data = fail.

Repository Artifact	Gap / Issue	Invention	Required Code Changes
`transition` logic	Transitions assumed executable if state loads.	Consequence-Indexed Gate; Transition Validator.	Insert a gate before each transition execution: compute equivalence class & consequences; require single outcome. Also enforce one disposition per state element (closure).
Error handling (exception messages)	Failures only raise generic errors, no receipt.	Typed Failure Receipt System.	Modify exception handlers to generate structured receipts (JSON) with predicate name, evidence, reopen flag. Store in state/ context.

8. Next Engineering Tasks & Tests

We recommend the following prioritized implementation tasks (with acceptance criteria and minimal tests):

1. **Implement Typed Admission Pipeline:**
2. *Action:* Modify state load to feed output into `AdmissionEngine.admit()`. Add admission step after reconstruction.
3. *AC:* If any admission predicate fails, the system must abort load; if all pass, execution continues with receipts logged.
4. *Test:* Create a snapshot with a bad signature: admission should fail (receipt lists `BYTESIGNATURE:FAIL`). Use a good signature: should admit.
5. **Define Typed Predicates & Receipts:**
6. *Action:* Encode predicates (BYTE_IDENTITY, SOURCE_EXISTS, etc.) as separate functions returning `(result, evidence, reopenRule, replayRule)`. Design a `Receipt` class.
7. *AC:* Each predicate must independently run and return a `Receipt` object, not just boolean.
8. *Test:* For each predicate type, feed a candidate state that *should* trigger PASS and FAIL. Verify receipt contains expected fields (name, status, context info).
9. **Enable Replay-Time Re-Admission:**

10. *Action:* Ensure all replays (e.g. recovery or chain replay) use the admission engine with up-to-date parameters.

11. *AC:* Changing external parameters (keyset, time) after snapshot creation should change admission result on replay.

12. *Test:* Take a valid snapshot. Revoke an authority key in the system, then replay: admission should now fail.

13. **Implement Consequence Identifiability Gate (Initial):**

14. *Action:* Before executing a transition, compute the current observation hash/class, derive consequences for two hypothetical worlds (or a small set), and compare.

15. *AC:* If two compatible worlds yield different outcomes, the transition must not execute. If identical, execution proceeds.

16. *Test:* Define a toy state with an ambiguous flag (e.g. “data is secret” vs “not secret” unknown). Schedule a transition that would do different things in each case. Verify execution is blocked (NOT_IDENTIFIABLE). Then add an observation resolving it and ensure execution proceeds.

17. **Implement Counterfactual Swap Assay:**

18. *Action:* For each claimed independent basis `B_R` and target label `L`, implement a swap test: flip `L` in two copies and re-derive consequences.

19. *AC:* If any swap changes the derived consequence, mark the basis as contaminated.

20. *Test:* Construct a basis with a circular dependency (B depends on the outcome). The swap test should catch it (independenceReceipt indicates failure).

21. **Add “Identifiability Status” States:**

22. *Action:* In the gating, output one of three statuses: IDENTIFIABLE, NOT_IDENTIFIABLE, INDEP_UNRESOLVED.

23. *AC:* The system distinguishes “we need more independence evidence” from “multiple consequences”.

24. *Test:* Force a scenario where the independence check hasn’t been run: should return INDEP_UNRESOLVED.

25. **Integrate Consequence-Tagging:**

26. *Action:* Tag state distinctions or transitions (e.g. mark payment vs inventory transitions). Group and limit execution per tag.

27. *AC:* The scheduler should only allow one transition per tag class at a time (example policy).

28. *Test:* Tag two equivalent “update balance” transitions identically; run both in parallel: system should serialize or drop one according to tag rules.

29. **Generate and Use Typed Failure Receipts:**

30. *Action:* Whenever admission or gating fails, emit a JSON receipt with the predicate name, evidence, and recommended next steps.

31. *AC:* The receipt must be machine-readable and influence the next action (e.g. block, retry after observation).

32. *Test:* Trigger an admission failure; inspect the receipt fields. Ensure the runtime pauses rather than continuing.

33. **Build Minimum Missing Observation Engine (Baseline):**

34. *Action:* Given a NOT_IDENTIFIABLE result, scan possible next observations (from a predefined set) and pick one that best reduces consequence entropy.

35. *AC:* Returns a candidate observation and expected outcome grouping.

36. *Test:* In a scenario with two possible worlds, the engine should pick an observation that distinguishes them (e.g. asks “is flag true?” to separate worlds).

37. **Implement Minimum Sufficient Observation:**

    - *Action:* Conversely, compute a subset of current observations sufficient to maintain identifiability. (Prune irrelevant data.)
    - *AC:* Identify minimal evidence needed so consequence stays invariant.
    - *Test:* Given full observation set, remove one redundant piece and show identifiability still holds.

38. **Reopening Propagation:**

    - *Action:* Given a changed predicate (e.g. expiry of data), traverse dependency graph to find and mark consequences that must reopen.
    - *AC:* Only tasks depending on the changed predicate are reopened.
    - *Test:* If predicate P fails, any transition whose basis included P should be flagged for review; others remain closed.

39. **Authority Graph Construction:**

    - *Action:* Build a directed graph of authorities (who granted what), track scopes/expiry.
    - *AC:* Authority checks become path existence in the graph.
    - *Test:* Simulate granting and revoking keys; queries like “is actor X authorized for action Y at time T?” should yield correct answer via graph lookup.

40. **Test False-Admission and False-Withholding:**

    - *Action:* Write tests ensuring system does not admit invalid state or unnecessarily withhold valid transition.
    - *AC:* One suite shows that valid equal-world transitions do execute (no false-positive blocking). Another shows invalid ones are blocked.
    - *Test:* Two compatible worlds with same consequence: transition should proceed. If consequence set has two values: block.

41. **Receipt Freezing:**

    - *Action:* Once stable, freeze a version of the receipt formats and circuit for inclusion in broader system (Brullama/Grax).
    - *AC:* Interfaces are finalized, further changes require version updates.
    - *Test:* Integration tests for other components consuming receipts should pass against the frozen format.

42. **Finalize Execution Gate:**

    - *Action:* Ensure that the identifiability check *actually prevents* transitions when NOT_IDENTIFIABLE (i.e., the gating is enforced, not advisory).
    - *AC:* There must be no path where a transition executes without identifiability.
    - *Test:* Run a workflow where an ambiguous transition exists; check that system never executes it until resolved.

9. Sample Claim Language (Top 3 Inventions)

(a) Typed State Admission Engine (Independent Claim)
“1. A computer-implemented method for authoritative state loading, comprising: receiving a reconstructed candidate state from persistent storage; for each of a plurality of **typed admission predicates** (comprising at least one of: data-integrity, source-authorship, tracking-membership, temporal-validity, authority-scope, interference-free, canon-eligibility), executing on the candidate state a verification algorithm that returns pass or fail and produces an evidence receipt; and only when all predicates return pass, constructing an *authoritative runtime state* from the candidate state and returning a success admission receipt including all evidence; otherwise aborting the loading.

Dependent 1: The method of claim 1, wherein the data-integrity predicate computes a cryptographic hash of the candidate’s bytes and compares to a stored digest.

Dependent 2: The method of claim 1, wherein the authority-scope predicate verifies a digital signature or key certificate in the candidate state.

Dependent 3: The method of claim 1, further comprising issuing a failure receipt when any predicate fails, the receipt including a classification code and a rule for state reopening or replay.”

(b) Consequence-Indexed Identifiability Gate (Independent Claim)
“1. A system for gating execution of state transitions based on partial observations, comprising: a processor

and memory storing program code which, when executed, perform: constructing an **observational-equivalence class** of possible worlds consistent with the current admitted state observations; for each world in the class, independently deriving a respective **consequence** of a proposed transition; and only if all derived consequences are identical, permitting the transition to execute with that consequence, else withholding the transition.

Dependent 1: The system of claim 1, wherein constructing the equivalence class includes grouping states by values of a defined observation vector and building a dependency graph of the admitted state.

Dependent 2: The system of claim 1, further performing a counterfactual test by swapping labels in the admitted state and re-deriving consequences to verify that none of the consequences depend causally on the transition's output (i.e., checking no path from outcome to its own basis).

Dependent 3: The system of claim 1, wherein upon withholding, an information-gain algorithm selects a next observation to discriminate among the multiple consequences."

(c) Consequence-Tag Selection Engine (Independent Claim)
"1. A computer system for selecting transitions using semantic tags, comprising: a data store and a processor executing instructions to: receive an admitted state annotated with one or more *consequence tags* attached to state elements or transitions; partition available transitions into groups by their associated tags; and select at most one transition per tag group for execution according to pre-defined policies, thereby preventing concurrent execution of multiple transitions with the same tag.

Dependent 1: The system of claim 1, wherein tags indicate categories such as "state-equivalence", "semantic-change", "delivery-confirmation", or "resource-discharge".

Dependent 2: The system of claim 1, wherein selecting comprises ordering the tag groups by priority and iterating through to pick one representative transition from each group.

Dependent 3: The system of claim 1, further comprising re-tagging transitions dynamically when the admitted state changes, and re-invoking the selection."

(Similar claim sets can be drafted for the Typed Failure Receipt System and other Tier 1 nuclei.)

Diagrams

Pipeline Flowchart: (Admission → Derivation → Consequence)

```
flowchart LR
    A[Repository Snapshot] --> B[Reconstruct Candidate State]
    B --> C[Replay-Time Re-Admission]
    C --> D[Typed Admission Engine (predicates/receipts)]
    D --> E[Authoritative Runtime State]
    E --> F[Consequence Derivation (for each possible world)]
    F --> G[Identifiability Gate (check single consequence)]
```

```
    G --> H[Chosen Consequence/Transitions]
    H --> I[Execute Transition]
```

Entity-Relationship (Key Receipts/Objects):

Object/Receipt	Fields (Key Attributes)	Related Entities
AdmissionReceipt	Observed candidate state ID; list of *(predicateName, status, evidence, reopenRule, replayRule)* tuples; overall pass/fail status.	References CandidateState, predicates. Used to build AuthoritativeState.
PredicateReceipt	predicateName (enum), result (PASS/FAIL), evidence (e.g. hash, signature), failureType, associated rules.	Part of AdmissionReceipt or FailureReceipt.
AdmittedState	Snapshot ID, combined predicate receipts, witness set of admitted bytes/objects.	Inputs to transition logic.
Observation	A subset of state variables known (e.g. {x=1, y=0}). Used to define equivalence classes.	Generates EquivalenceClass of possible worlds.
EquivalenceClass (E)	Identifier (e.g. hash of observations); set of possible worlds (implicit).	Input to Identifiability logic.
ConsequenceBasis (B)	Dependencies used for deriving consequence (e.g. a set of source statements).	Provided to each derive() call.
IndependenceReceipt	basisID, label (L), intervention info (e.g. swap performed), before/after consequences, status (INDEP/ CONTAMINATED).	Produced by swap assay; used by IdentifiabilityGate.
IdentifiabilityReceipt	observationID, consequenceSet, status (IDENTIFIABLE/NOT/UNRESOLVED).	Final output of identifiability test; controls execution.
FailureReceipt	Trigger (predicate or gate name), evidence, errorCode, reopenNeeded (bool), suggestedAction.	Logged or attached to state; can itself be admitted in replay.

(ER table illustrates how receipts link predicates, states, and transitions.)

References

- Bauer *et al.*, “Runtime Verification for LTL and TLTL,” *ACM Trans. Softw. Eng. Methodol.*, introducing three-valued (true/false/?_inconclusive) semantics for partial observations【2†L15-L18】.

- W3C PROV Data Model (PROV-DM)【23†L64-L67】: domain-agnostic provenance framework (no built-in “admission”).
- In-toto: “framework to secure supply chain integrity… transparent [verification] of steps performed”【24†L8-L11】.
- Pearl (2000) et al., on counterfactual identifiability – e.g. LATE not identifiable from data【9†L43-L49】.
- USPTO MPEP §2106.04(d)(1): emphasize computing device improvements (“improves functioning of a computer”)【20†L7-L10】.
- Patent attorney guidance: “best practice… describe how software invention improves underlying computing device (speed, efficiency, etc.)”【20†L1-L4】【18†L23-L27】.
- See [capsule.verification.json] which explicitly marks “authority:NONE” etc., showing current gap (no state admission). *(Repository context)*.
